

DBMS UNIT-I

Two marks questions:

1. What is database? Define DBMS.

Ans: A database is a collection of related data. For example: names, telephone numbers, address etc. A database management system (DBMS) is a collection of programs that enables users to create and maintain a database. The DBMS hence is a general purpose software system that facilitates the process of defining, constructing and manipulating database for various applications.

2. What do you mean by defining, constructing, manipulating and sharing a database?

- Defining database involves specifying the data types, structures and constraints for the data to be stored in the database.
- Constructing the database is the process of storing the data itself on some storage medium that is controlled by the DBMS.
- Manipulating a database includes functions such as querying, updating and generating reports on the data stored in the database.

3. Define data abstraction. Give any two levels of abstraction

Ans: Data Abstraction is a process of hiding unwanted or irrelevant details from the end user. It provides a different view and helps in achieving data independence which is used to enhance the security of data.

Levels of Data Abstraction in DBMS

- Physical or Internal Level
- Logical or Conceptual Level
- View or External Level

4. Define query and transaction Ans:

- A query in a database is a request for information from a database management system (DBMS), which is the software program that maintains data. Users can make a query to retrieve data or change information in a database, such as adding or removing data.
- A transaction can be defined as a group of tasks. A single task is the minimum processing unit which cannot be divided further.

5. List the phases of database application.

Ans: The database development life cycle includes eight steps that help guide us through the creation of a new database. The steps are planning, requirement gathering, conceptual design, logical design, physical design, construction, implementation and rollout, and finally ongoing support.

6. List the characteristics of Database approach.

Ans: Database approach has the following characteristics

- Self- Describing Nature of a Database System
- Insulation between Program Data and Data Abstraction

- Support Multiple Views of the Data
- Sharing of Data and Multi-user Transaction Processing

7. Give the disadvantages of file processing system.

Ans: Disadvantages of File Processing System:

- Slow access time – Direct access of files is very difficult and one needs to know the entire hierarchy of folders to get to a specific file. This involves a lot of time.
- Presence of redundant data – The same data can be present in two or more files which takes up more disc space.
- Inconsistent Data – Due to data redundancy, same data stored at different places might not match to each other.
- Data Integrity Problems – The data present in the database should be consistent and correct. To achieve this, the data should satisfy certain constraints.

8. Define Metadata. Give an example

Ans: Metadata is defined as the data providing information about one or more aspects of the data; it is used to summarize basic information about data that can make tracking and working with specific data easier.

For example, author, date created, date modified and file size.

9. What do you mean by program data independence and program-operation independence?

- The structure of data files is embedded in the application programs, so any changes to the structure of a file do not require changing all programs that access the database. The structure of data files is stored in the DBMS catalog separately from the access programs. This property is called program-data independence.
- An operation (also called a function or method) is specified in two parts. The interface (or signature) of an operation includes the operation name and the data types of its arguments (or parameters). The implementation (or method) of the operation is specified separately and can be changed without affecting the interface. User application programs can operate on the data by invoking these operations through their names and arguments, regardless of how the operations are implemented. This may be termed program-operation independence.

10. Define Data Model. Give the categories of data model.

Ans: Data models are visual representations of an enterprise's data elements and the connections between them. By helping to define and structure data in the context of relevant business processes, models support the development of effective information systems.

There are three types of data models: dimensional, relational, and entity relational. These models follow three approaches: conceptual, logical, and physical.

11. Mention the following:

- Actors on the scene.**
- Workers behind the scene.**

Ans: One person typically defines, constructs, and manipulates the database, and there is no sharing. However, many persons are involved in the design, use, and maintenance of a large database with hundreds of users. In this section we identify the people whose jobs involve the

day-to-day use of a large database; we call them the "actors on the scene". We consider people who may be called "workers behind the scene"-those who work to maintain the database system environment but who are not actively interested in the database itself.

14. What is redundancy? Which problems may arise while storing redundant data?

Redundancy means having multiple copies of same data in the database. This problem arises when a database is not normalized. Suppose a table of student details attributes are: student Id, student name, college name, college rank, course opted.

Student_ID	Name	Contact	College	Course	Rank
100	Himanshu	7300934851	GEU	Btech	1
101	Ankit	7900734858	GEU	Btech	1
102	Aysuh	7300936759	GEU	Btech	1
103	Ravi	7300901556	GEU	Btech	1

As it can be observed that values of attribute college name, college rank, course is being repeated which can lead to problems. Problems caused due to redundancy are: Insertion anomaly, Deletion anomaly, and Updating anomaly.

15. List the limitations/disadvantages of DBMS.

- o More Costly. Creating and managing a database is quite costly.
- o High Complexity. A Database Management System is quite complex as it involves creating, modifying and editing a database.
- Disadvantages: o Increased Cost. o Complexity. o Database Failure.

16. Who is a DBA? What are the responsibilities of a DBA?

Ans: A database administrator (DBA) is a person or group in charge of implementing DBMS in an organization. The DBA job requires a high degree of technical expertise. DBA consists of a team of people rather than just one person The responsibilities of DBA are as follows –

- Makes the decision concerning the content of the database.
- Plans the storage structure and access strategy.
- Provides the support to the users.
- Defines the security and integrity checks.

17. Who are database designers? What are their responsibilities?

Ans: Database designers are responsible for identifying the data to be stored in the database and for choosing the appropriate structure to represent and store this data. These tasks are usually undertaken before the database is actually implemented and populated with data. It is the responsibility of the database designers to communicate with all database users, in order to understand their requirements, and to come up with a design that meets these requirements. In many cases, the designers are on the staff of the DBA and may be assigned other staff responsibilities after the database design is completed.

18. What is the responsibility of backup and recovery subsystem?

Ans: A DBMS must provide facilities for recovering from hardware or software failures. The backup and recovery subsystem of the DBMS is responsible for recovery. For example, if the computer system fails in the middle of a complex update transaction, the recovery subsystem is responsible for making sure that the database is restored to the state it was in before the transaction started executing. Alternatively, the recovery subsystem could ensure that the transaction is resumed from the point at which it was interrupted so that its full effect is recorded in the database. Disk backup is also necessary in case of a catastrophic disk failure.

19. What are the advantages of using a DBMS?

- Controlling Redundancy
- Restricting Unauthorized Access
- Providing Persistent Storage for Program Objects and Data Structures
- Providing Multiple User Interfaces

20. What does the isolation and atomicity property ensure?

- Atomicity: By this, we mean that either the entire transaction takes place at once or doesn't happen at all. Each transaction is considered as one unit and either runs to completion or is not executed at all.
- Isolation: This property ensures that multiple transactions can occur concurrently without leading to the inconsistency of the database state. Transactions occur independently without interference.

21. Define entity, attribute and relationship.

- Entity represents a real-world object or concept, such as an employee or a project from the mini world that is described in the database.
- Attribute represents some property of interest that further describes an entity, such as the employee's name or salary.
- Relationship among two or more entities represents an association among the entities, for example, a works-on relationship between an employee and a project.

22. What is database schema, schema diagram?

Ans: The description of a database is called the database schema, which is specified during database design and is not expected to change frequently. Most data models have certain conventions for displaying schemas as diagrams. A displayed schema is called a schema diagram.

23. What do you mean by database state? When do you say the database state is valid?

Ans: The data in the database at a particular moment in time is called a database state or snapshot. It is also called the current set of occurrences or instances in the database. The DBMS is partly responsible for ensuring that every state of the database is a valid state—that is, a state that satisfies the structure and constraints specified in the schema.

24. Define the terms Internal, external and conceptual schema (any two).

1. The internal level has an internal schema, which describes the physical storage structure of the database. The internal schema uses a physical data model and describes the complete details of data storage and access paths for the database.
2. The conceptual level has a conceptual schema, which describes the structure of the whole database for a community of users. The conceptual schema hides the details of physical storage structures and concentrates on describing entities, data types, relationships, user operations, and constraints.
3. The external or view level includes a number of external schemas or user views. Each external schema describes the part of the database that a particular user group is interested in and hides the rest of the database from that user group.

25. What do you mean by Logical Data Independence?

Ans: Logical data independence is the capacity to change the conceptual schema without having to change external schemas or application programs. We may change the conceptual schema to expand the database (by adding a record type or data item), to change constraints, or to reduce the database (by removing a record type or data item). In the last case, external schemas that refer only to the remaining data should not be affected.

26. What do you mean by physical data Independence?

Ans: Physical data independence is the capacity to change the internal schema without having to change the conceptual schema. Hence, the external schemas need not be changed as well. Changes to the internal schema may be needed because some physical files were reorganized—for example, by creating additional access structures—to improve the performance of retrieval or update. If the same data as before remains in the database, we should not have to change the conceptual schema.

27. What is the purpose of storage manager?

Ans: Storage Manager is a program that provides an interface between the data stored in the database and the queries received. It is also known as Database Control System. It maintains the consistency and integrity of the database by applying the constraints and executing the DCL statements.

28. List the functions of runtime database processor.

- The runtime database processor executes (1) the privileged commands, (2) the executable query plans, and (3) the canned transactions with runtime parameters.
- It works with the system catalog and may update it with statistics.
- It also works with the stored data manager, which in turn uses basic operating system services for carrying out low-level input/output operations between the disk and main memory.
- It also handles other aspects of data transfer, such as management of buffers in the main memory.

29. Discuss about Data Definition language?

Ans: Data Definition Language (DDL) is a subset of SQL. It is a language for describing data and its relationships in a database. You can generate DDL in a script for database objects to: Keep a snapshot of the database structure.

30. Discuss about Data Definition language?

Ans: Data Definition Language (DDL) is a subset of SQL. It is a language for describing data and its relationships in a database. You can generate DDL in a script for database objects to: Keep a snapshot of the database structure.

31. List the functions of Database System Utilities Ans: The functions of a DBMS are:

1. concurrency
2. security
3. backup and recovery
4. integrity and data descriptions.

32. Define a) meta-data b) data dictionary

Ans: a) meta-data : Metadata is defined as the data providing information about one or more aspects of the data. it is used to summarize basic information about data that can make tracking and working with specific data easier. Some examples include: Means of creation of the data.

b) data dictionary: A Data Dictionary is an integral part of a database. It holds the information about the database and the data that it stores called as metadata. A meta data is the data about the data. It is the self-describing nature of databases. It holds the information about each data element in the database.

33. List various types of attributes?

1. Simple
2. Composite
3. Single-valued
4. Multi-valued
5. Derived attribute

34. Explain data model and list the types of data model used?

A data model can be defined as an integrated collection of concepts describing and manipulating data, as well as constraints on the data within an organization.

Types of Data Models in DBMS

- ☐ Hierarchical Model. This concept uses a hierarchical tree structure to organise the data. ...
- ☐ Network Model.
- ☐ Entity-Relationship Model.
- ☐ Relational Model.
- ☐ Object-Oriented Data Model.
- ☐ Object-Relational Data Model.
- ☐ Flat Data Model.
- ☐ Semi-Structured Data Model.

35. Define Single-Valued versus Multivalued Attributes, give example

Single-valued attribute : The attribute which takes up only a single value for each entity instance is a single-valued attribute. Example: The age of a student.

Multi-valued attribute: The attribute which takes up more than a single value for each entity instance is a multi-valued attribute. Example: Phone number of a student: Landline and mobile.

36. Define Stored versus Derived Attributes, give example

Stored attributes: are those attributes that are stored in the physical database for e.g date of birth.

Derived attributes: are those attributes that are not present in the physical database, but can be derived from the attributes present in the database. For e.g. age can be derived from date of birth.

37. Define Composite versus Simple (Atomic) Attributes, give example

Simple attributes: The attributes that cannot be divided into sub-parts are called simple attributes.

Composite attributes: The attributes which can be divided into sub-parts are called composite attributes.

Example

- Balance, account number are simple attributes
- Name, address is called composite attributes.

39. Define the terms i) Entity set ii) Relationship set

(i) Entity set: An entity set in DBMS is a set that collectively represents a group of entities of a similar type. Example: An entity set of cars, an entity set of bank accounts, etc.

(ii) Relationship set: A collection of various relationships that belongs to the same relationship type is called a relationship set.

Example: consider the two entity sets customer and account.

40. Define the terms i) Key attribute ii) Value set(Domain)

(i) key attribute: Key allows us to identify a set of attributes that make it sufficient to distinguish entities from each other.

Example:

- stdNo, name}
- {stdNo, address}
- {branch, section}
- {stdNo} etc.

(ii) Value Sets (Domains)of Attributes:

This value set is also called domain. The domain of an attribute is the collection of all possible values an attribute can have. The value set specifies the set of values that may be assigned for each individual entity. Example: we can specify the value set for designation attribute as <“PM”, “Assit”, “DM”, “SE”>.

41. What do you mean by relationship type and relationship instances?

•Relationship type: is a type of association that can exist between two different (or same) entity types.

For example: a person can manage another person, or an organization can legally own another organization.

- Relation instance – A finite set of tuples in the relational database system represents relation instance. Relation instances do not have duplicate tuples.

42. Define “degree of relationship type”.

A degree of relationship represents the number of entity types that associate in a relationship.

For example, we have two entities, one is a student and the other is a bag and they are connected with the primary key and foreign key. So, here we can see that the degree of relationship is 2 as 2 entities are associating in a relationship.

43. What do you mean by role names in the relationship? Give example.

The role name signifies the role that a participating entity from the entity type plays in each relationship instance, and helps to explain what the relationship means. For example, in the WORKS_FOR relationship type, EMPLOYEE plays the role of employee or worker and DEPARTMENT plays the role of department or employer.

44. What does the cardinality ratio specify? Which are the possible cardinality ratios for binary relationship types

In ER model, a relationship is an association among entities (records) of one or more entity sets. Cardinality ratio or mapping cardinalities is a concept that describes binary relationship set (a relationship that connects two entity sets) and its types. It is about the maximum number of entities of one entity set that are associated with the maximum number of entities of the other entity set. The possible cardinality ratios of binary relationship types are **(1:1, 1:M, M:1, M:M)**.

One to one relationship, One to many relationship, Many to one relationship, Many to Many relationship

45. Define the terms

i) Total participation

Total Participation – Each entity is involved in the relationship. Total participation is represented by double lines.

ii) Partial Participation

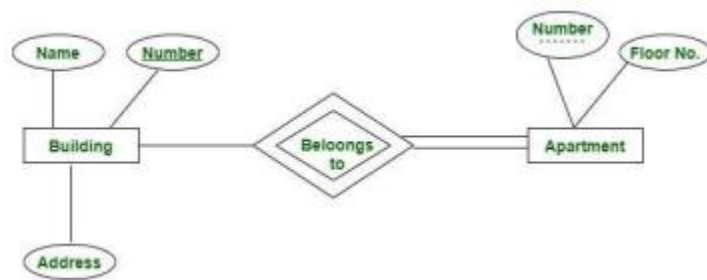
Partial participation – Not all entities are involved in the relationship. Partial participation is represented by single lines.

46. Define weak and strong entity types?

The entity set which does not have sufficient attributes to form a primary key is called as Weak entity set. An entity set that has a primary key is called as Strong entity set. Consider an entity set Payment which has three attributes: payment_number, payment_date and payment_amount.

47. What is partial key? Give example.

Ans: Partial key is a key using which all the records of the table can not be identified uniquely. However, a bunch of related tuples can be selected from the table using the partial key.



48. Who is Database Administrator?

A Database Administrator (DBA) in Database Management System (DBMS) is an IT professional who works on creating, maintaining, querying, and tuning the database of the organization. They are also responsible for maintaining data security and integrity.

49. Definition of system designer, tool developer, operator & maintenance personnel.

50. What is meant by restricting unauthorized access?

Restricting unauthorized access DBMS provides security and authorization so that some users are not authorized to access all data in the database. As DBMS facilitates the use of multiple views, this ensures that only authorized people can access the view and not everyone.

51. What is deductive database system?

A deductive database is a database system that makes conclusions about its data based on a set of well-defined rules and facts. This type of database was developed to combine logic programming with relational database management systems.

52. Why storage definition language (SDL) is used?

DBMS supports many languages out of which (SDL) is one of them. SDL stands for Storage Definition Language. SDL matter is almost anything that's not specified by SQL standard. It is different in every DBMS which specifies anything to do with how or where data in relevant table is stored.

53. Why interfaces for DBA are used?

Interfaces for DBA – Most database system contains privileged commands that can be used only by the DBA's staff. These include commands for creating accounts, setting system parameters, granting account authorization, changing a schema, reorganizing the storage structures of a databases.

3 OR MORE MARKS

1. What are the characteristics of DBMS? Explain

The following are some of the most important and well-known characteristics of DBMS

❖ **Real World Entity**

- One of the most important and easy-to-understand characteristics of DBMS is that it is realistic.
- The DBMS has been designed in such a way that it can cater to the needs of huge business organizations and can store large data with efficient operations on them. The entities in the database look like the real world entities.

❖ **Self Explaining Nature**

- A DBMS contains one database and along with that it also contains metadata about that one database.
- Metadata is the data about data. For example: In a DBMS for a particular School, the total Number of rows in the database and what is the name of each column of the database table, and all such information about the data is metadata.
- So, the combination of this database and metadata leaves no questions in anyone's mind as the DBMS becomes self-explanatory. This is because the database has all the information in a structured format and if anyone has any doubts or questions regarding how the database is designed, they may look into the metadata.

❖ **Concurrent Access without Anomalies**

- Multiple users can access the database at the same time without any anomalies (problems: like 2 people trying to access the same data might restrict one from accessing it).
- Thus, we see that concurrent access is such a great characteristic of DBMS.

❖ **Stores Any Kind of Structured Data**

- A database can store any kind of structured data.
- In most of the books or websites, you will see examples of students or employees as these are easy to explain but the very important fact is that the DBMS is not limited to storing only these kinds of information.
- DBMS can store practically any data that exists in the world and is structured and this is yet another very important characteristic because we need to work on every kind of data present.

❖ **Ease of Access (The DBMS Queries)**

- Before DBMS, the conventional file system (i.e. files and folders) was used to store complex and large data.
- Searching for a Name in thousands of Students was a very difficult task. This is because manual searching is done in a conventional file system for a particular data. However, DBMS provides ease of access to the data inside the database.
- We can run a search query to find any data and the process is way faster than manual searching and is more reliable.
- The CRUD (Create, Read, Update & Delete) operations are very easy to perform with a database because of the DBMS queries.

2. Explain the different categories of End Users in DBMS

End users are the people whose jobs require access to the database for querying, updating and generating reports. There are several categories of end-users.

- **Casual End Users:** They occasionally access the database, but they may need different information each time. They use a sophisticated database query language to specify their requests and are typically middle or high-level managers.
-

-
- **Parametric End Users:** Their main job function revolves around constantly querying and updating the database, using standard types of queries. These transactions are called as canned transactions that have been carefully programmed and tested. For example, Reservation clerks for airline or hotels check the availability for a given request and make reservations.
 - **Sophisticated End Users:** This category includes engineers, scientists business analysts etc. who thoroughly familiarize themselves with the facilities of DBMS so as to implement their applications to meet their complex requirements.
 - **Stand-alone End Users:** They maintain personal databases by using ready-made program packages that provide easy-to-use menu or graphics-based interfaces.

3. Write a short note on

1. DBA

2. Database Designers

Database Administrators: In any organization where many persons use the same resources, there is a need for a chief administrator to oversee and manage the resources. In a database environment, the primary resource is the database itself and the secondary resource is the DBMS and related software. Administering these resources is the responsibility of the database administrator (DBA). The DBA is responsible for authorizing access to the database, for coordinating and monitoring its use and for acquiring software and hardware resources as needed.

Database Designers: Database designers are responsible for identifying the data to be stored in the database and for choosing the appropriate structure to represent and store this data. These tasks are usually undertaken before the database is actually implemented and populated with data. It is the responsibility of the database designers to communicate with all database users, in order to understand their requirements, and to come up with a design that meets these requirements.

4. Explain the different categories of workers those are associated with the design, development and operation of the DBMS.

Workers behind the Scene

The persons who are associated with the design, development and operation of the DBMS software and system environment are typically not interested in the database itself. These persons are called as “workers behind the scene”. They include the following categories:

- ♣ DBMS system designers and implementers are persons who design and implement the DBMS modules and interfaces as a software package. A DBMS is a very complex software system that consists of many components, or modules, including modules for implementing the catalog, query language, interface processors, data access, concurrency control, recovery and security.
 - ♣ Tool developers include persons who design and implement tools – the software packages that facilitate database system design and use and that help improve performance. Tools include packages for database design, performance monitoring, natural language or graphical interfaces, prototyping, simulation, and test data generation.
 - ♣ Operators maintenance personnel are the system administration personnel who are
-

responsible for the actual running and maintenance of the hardware and software environment for the database system.

5. Explain any four advantages of DBMS

Controlling Redundancy:

Redundancy is nothing but storing same data repeatedly. In database approach a huge repository of data is stored and users are allowed to access the data which is relevant to them there by controlling the redundancy.

Restricting Unauthorized Access:

When multiple users share a database, it is likely that some users will not be authorized to access all information in the database and hence only authorized persons are allowed to access the data. In addition to this, some users may be permitted only to retrieve data, where as others are allowed both to access, retrieve and update. Hence the type of access operation must be controlled. and DBMS should provide a security and authorization subsystem to specify access restrictions to various users.

Providing Persistent Storage for Program Objects and Data Structures :

Databases can be used to provide persistent storage for program objects and data structures so that the user of the database can store any amount of data in the database for further use

Providing Multiple User Interfaces:

Because many types of users with varying levels of technical knowledge use a database, a DBMS should provide a variety of user interfaces. These include query languages for casual users; programming language interfaces for application programmers; forms and command codes for parametric users; and menu-driven interfaces and natural language interfaces for stand-alone users. Both forms – style interfaces and menu – driven interfaces are commonly known as graphical user interfaces (GUIs).

Representing Complex Relationships among Data:

A database may include numerous varieties of data that are interrelated in many ways. A DBMS must have the capability to represent variety of complex relationships among the data as well as to retrieve and update related data easily and efficiently.

Enforcing Integrity Constraints:

Most database applications have certain integrity constraints that must be enforced on the data to be stored in the table. The data that satisfies the integrity constraint will be stored in the table for the further use, the data that fails to satisfy the integrity constraints will be rejected to ensure consistency.

Providing Backup and Recovery:

A DBMS must provide facilities for recovering from hardware and software failures. The backup and recovery subsystem of the DBMS is responsible for recovery.

6. Explain the different categories of data models.

The Data Model gives us an idea of how the final system would look after it has been fully implemented. It specifies the data items as well as the relationships between them. In a database management system, data models are often used to show how data is connected, stored, accessed,

and changed. We portray the information using a set of symbols and language so that members of an organisation may understand and comprehend it and then communicate.

Types of Data Models in DBMS

Hierarchical Model

This concept uses a hierarchical tree structure to organise the data. The hierarchy begins at the root, which contains root data, and then grows into a tree as child nodes are added to the parent node.

Network Model

The main difference between this model and the hierarchical model is that any record can have several parents in the network model. It uses a graph instead of a hierarchical tree.

Entity-Relationship Model

The real-world problem is depicted in visual form in this model to make it easier for stakeholders to comprehend. The ER diagram also makes it very simple for developers to comprehend the system.

Relational Model

The data in this model is kept in the form of a table that is two-dimensional. All of the data is kept in the form of rows and columns. Tables are the foundation of a relational paradigm.

Object-Oriented Data Model

Both the data and the relationship are contained in a single structure that is known as an object in this model. We can now store audio, video, pictures, and other types of data in databases, which was previously impossible with the relational approach (Although you can store video and audio in relational DB, it is advised not to store them in the relational database).

Object-Relational Data Model

It is a hybrid of relational and object-oriented models. This model was developed to bridge the gap between the object-oriented and relational models.

Flat Data Model

It's a straightforward model in which the DB is depicted as a table with rows and columns.

Semi-Structured Data Model

The relational model has evolved into the semi-structured model. In this model, we can't tell the difference between data and schema.

Associative Data Model

It is a model in which the data is separated into two sections. Everything that has its own existence is referred to as an entity, and the relationships between these entities are referred to as associations. Items and links are two types of data that are separated into two components.

Context Data Model

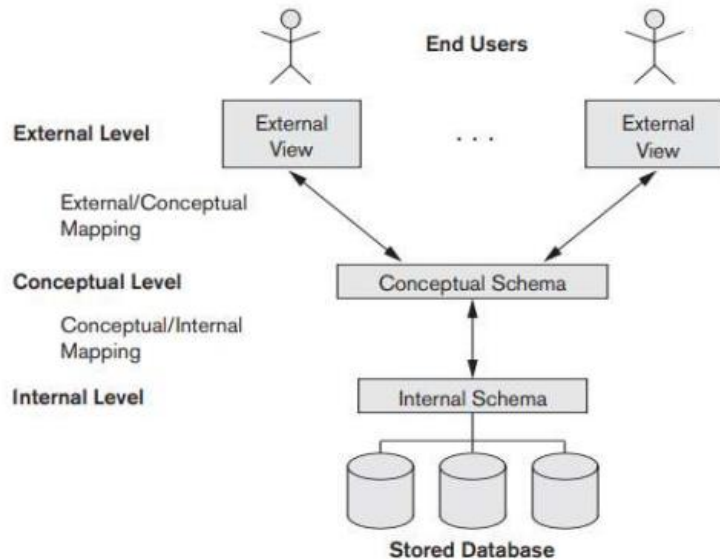
The Context Data Model is made up of various models. This includes models such as network models and relational models, among others.

8. Explain three schema architecture of database with a neat diagram.

- ☐ The internal level has an internal schema, which describes the physical storage structure of the database. The internal schema uses a physical data model and describes the complete details of data storage and access paths for the database.
 - ☐ The conceptual level has a conceptual schema, which describes the structure of the whole database for a community of users. The conceptual schema hides the details of physical storage structures and concentrates on describing entities, data types, relationships, user operations, and constraints. Usually, a representational data model is used to describe the conceptual schema
-

when a database system is implemented. This implementation conceptual schema is often based on a conceptual schema design in a high-level data model.

□ The external or view level includes a number of external schemas or user views. Each external schema describes the part of the database that a particular user group is interested in and hides the rest of the database from that user group. As in the previous level, each external schema is typically implemented using a representational data model, possibly based on an external schema design in a high-level data model.



9. What is data Independence? Explain their types.

□ Logical data independence is the capacity to change the conceptual schema without having to change external schemas or application programs. We may change the conceptual schema to expand the database (by adding a record type or data item), to change constraints, or to reduce the database (by removing a record type or data item). In the last case, external schemas that refer only to the remaining data should not be affected. After the conceptual schema undergoes a logical reorganization, application programs that reference the external schema constructs must work as before. Database System Concepts and Architecture Changes to constraints can be applied to the conceptual schema without affecting the external schemas or application programs.

□ Physical data independence is the capacity to change the internal schema without having to change the conceptual schema. Hence, the external schemas need not be changed as well. Changes to the internal schema may be needed because some physical files were reorganized—for example, by creating additional access structures—to improve the performance of retrieval or update. If the same data as before remains in the database, we should not have to change the conceptual schema. Generally, physical data independence exists in most databases and file environments where physical details such as the exact location of data on disk, and hardware details of storage encoding, placement, compression, splitting, merging of records, and so on are hidden from the user.

□ On the other hand, logical data independence is harder to achieve because it allows structural and constraint changes without affecting application programs—a much stricter requirement. Whenever we have a multiple-level DBMS, its catalog must be expanded to include information

on how to map requests and data among the various levels. The DBMS uses additional software to accomplish these mappings by referring to the mapping information in the catalog.

10. What are the different database languages? Explain with example.

□ In many DBMSs where no strict separation of levels is maintained, one language, called the data definition language (DDL), is used by the DBA and by database designers to define both schemas. The DBMS will have a DDL compiler whose function is to process DDL statements in order to identify descriptions of the schema constructs and to store the schema description in the DBMS catalog.

□ Another language, the storage definition language (SDL), is used to specify the internal schema. The mappings between the two schemas may be specified in either one of these languages. In most relational DBMSs today, there is no specific language that performs the role of SDL. Instead, the internal schema is specified by a combination of functions, parameters, and specifications related to storage. These permit the DBA staff to control indexing choices and mapping of data to storage.

□ For a true three-schema architecture, we would need a third language, the view definition language (VDL), to specify user views and their mappings to the conceptual schema, but in most DBMSs the DDL is used to define both conceptual and external schemas. In relational DBMSs, SQL is used in the role of VDL to define user or application views as results of predefined queries.

□ Once the database schemas are compiled and the database is populated with data, users must have some means to manipulate the database. Typical manipulations include retrieval, insertion, deletion, and modification of the data. The DBMS provides a set of operations or a language called the data manipulation language (DML) for these purposes. Storage definition is typically kept separate, since it is used for defining physical storage structures to fine-tune the performance of the database system, which is usually done by the DBA staff. A typical example of a comprehensive database language is the SQL relational database language which represents a combination of DDL, VDL, and DML, as well as statements for constraint specification, schema evolution, and other features.

□ Many DBMSs allow high-level DML statements either to be entered interactively from a display monitor or terminal or to be embedded in a general-purpose programming language. In the latter case, DML statements must be identified within the program so that they can be extracted by a precompiler and processed by the DBMS. A lowlevel or procedural DML must be embedded in a general-purpose programming language. This type of DML typically retrieves individual records or objects from the database and processes each separately.

□ Low-level DMLs are also called record-at-a-time DMLs because of this property. DL/1, a DML designed for the hierarchical model, is a low-level DML that uses commands such as GET UNIQUE, GET NEXT, or GET NEXT WITHIN PARENT to navigate from record to record within a hierarchy of records in the database.

□ Highlevel DMLs, such as SQL, can specify and retrieve many records in a single DML statement; therefore, they are called set-at-a-time or set-oriented DMLs. A query in a high-level DML often specifies which data to retrieve rather than how to retrieve it; therefore, such languages are also called declarative.

-
- Whenever DML commands, whether high level or low level, are embedded in a general-purpose programming language, that language is called the host language and the DML is called the data sublanguage.
 - On the other hand, a high-level DML used in a standalone interactive manner is called a query language. In general, both retrieval and update commands of a high-level DML may be used interactively and are hence considered part of the query language.
 - Casual end users typically use a high-level query language to specify their requests, whereas programmers use the DML in its embedded form.

11. What are the User friendly interfaces provided by DBMS? Explain any four.

- **Menu-Based Interfaces for Web Clients or Browsing:** These interfaces present the user with lists of options (called menus) that lead the user through the formulation of a request. Menus do away with the need to memorize the specific commands and syntax of a query language; rather, the query is composed step-by-step by picking options from a menu that is displayed by the system. Pull-down menus are a very popular technique in Web-based user interfaces. They are also often used in browsing interfaces, which allow a user to look through the contents of a database in an exploratory and unstructured manner.
 - **Forms-Based Interfaces:** A forms-based interface displays a form to each user. Users can fill out all of the form entries to insert new data, or they can fill out only certain entries, in which case the DBMS will retrieve matching data for the remaining entries. Forms are usually designed and programmed for naive users as interfaces to canned transactions. Many DBMSs have forms specification languages, which are special languages that help programmers specify such forms. SQL*Forms is a form-based language that specifies queries using a form designed in conjunction with the relational database schema. Oracle Forms is a component of the Oracle product suite that provides an extensive set of features to design and build applications using forms.
 - **Graphical User Interfaces:** A GUI typically displays a schema to the user in diagrammatic form. The user then can specify a query by manipulating the diagram. In many cases, GUIs utilize both menus and forms. Most GUIs use a pointing device, such as a mouse, to select certain parts of the displayed schema diagram.
 - **Natural Language Interfaces:** These interfaces accept requests written in English or some other language and attempt to understand them. A natural language interface usually has its own schema, which is similar to the database conceptual schema, as well as a dictionary of important words. The natural language interface refers to the words in its schema, as well as to the set of standard words in its dictionary, to interpret the request. If the interpretation is successful, the interface generates a high-level query corresponding to the natural language request and submits it to the DBMS for processing; otherwise, a dialogue is started with the user to clarify the request. The capabilities of natural language interfaces have not advanced rapidly. They use predefined indexes on words and use ranking functions to retrieve and present resulting documents in a decreasing degree of match. Such “free form” textual query interfaces are not yet common in structured relational or legacy model databases, although a research area called keyword-based querying has emerged recently for relational databases.
 - **Speech Input and Output:** Limited use of speech as an input query and speech as an answer to a question or result of a request is becoming commonplace. Applications with limited vocabularies such as inquiries for telephone directory, flight arrival/departure, and credit card account information are allowing speech for input and output to enable customers to access this information. The speech input is detected using a library of predefined words and used to set up
-

the parameters that are supplied to the queries. For output, a similar conversion from text or numbers into speech takes place.

□ **Interfaces for Parametric Users:** Parametric users, such as bank tellers, often have a small set of operations that they must perform repeatedly. For example, a teller is able to use single function keys to invoke routine and repetitive transactions such as account deposits or withdrawals, or balance inquiries. Systems analysts and programmers design and implement a special interface for each known class of naive users. Usually a small set of abbreviated commands is included, with the goal of minimizing the number of keystrokes required for each request.

12. Write a note on Database system utilities.

□ **Loading:** A loading utility is used to load existing data files—such as text files or sequential files—into the database. Usually, the current (source) format of the data file and the desired (target) database file structure are specified to the utility, which then automatically reformats the data and stores it in the database. Some vendors are offering products that generate the appropriate loading programs, given the existing source and target database storage descriptions (internal schemas). Such tools are also called conversion tools.

□ **Backup:** A backup utility creates a backup copy of the database, usually by dumping the entire database onto tape or other mass storage medium. The backup copy can be used to restore the database in case of catastrophic disk failure. Incremental backups are also often used, where only changes since the previous backup are recorded. Incremental backup is more complex, but saves storage space.

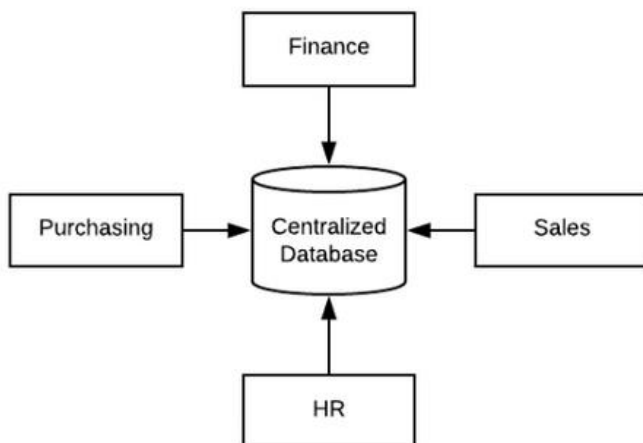
□ **Database storage reorganization:** This utility can be used to reorganize a set of database files into different file organizations, and create new access paths to improve performance.

□ **Performance monitoring:** Such a utility monitors database usage and provides statistics to the DBA. The DBA uses the statistics in making decisions such as whether or not to reorganize files or whether to add or drop indexes to improve performance.

□ Other utilities may be available for sorting files, handling data compression, monitoring access by users, interfacing with the network, and performing other functions.

13. Write a short note on Centralized DBMSs Architecture.

A centralized database is stored at a single location such as a mainframe computer. It is maintained and modified from that location only and usually accessed using an internet connection such as a LAN or WAN. The centralized database is used by organisations such as colleges, companies, banks etc.



As can be seen from the above diagram, all the information for the organisation is stored in a single database. This database is known as the centralized database.

Advantages

- The data integrity is maximised as the whole database is stored at a single physical location. This means that it is easier to coordinate the data and it is as accurate and consistent as possible.
- The data redundancy is minimal in the centralised database. All the data is stored together and not scattered across different locations. So, it is easier to make sure there is no redundant data available.
- Since all the data is in one place, there can be stronger security measures around it. So, the centralised database is much more secure.
- Data is easily portable because it is stored at the same place.
- The centralized database is cheaper than other types of databases as it requires less power and maintenance.
- All the information in the centralized database can be easily accessed from the same location and at the same time.

Disadvantages

- Since all the data is at one location, it takes more time to search and access it. If the network is slow, this process takes even more time.
- There is a lot of data access traffic for the centralized database. This may create a bottleneck situation.
- Since all the data is at the same location, if multiple users try to access it simultaneously it creates a problem. This may reduce the efficiency of the system.
- If there are no database recovery measures in place and a system failure occurs, then all the data in the database will be destroyed.

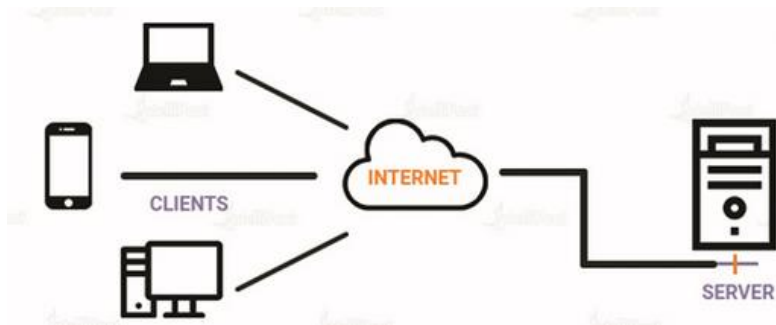
14. Write a short note on Client/Server Architectures.

The client/server architecture was developed to deal with computing environments in which a large number of PCs, workstations, file servers, printers, data-base servers, Web servers, e-mail servers, and other software and equipment are connected via a network. The idea is to define specialized servers with specific functionalities.

Web servers or e-mail servers also fall into the specialized server category. The resources provided by specialized servers can be accessed by many client machines. The client machines provide the user with the appropriate interfaces to utilize these servers, as well as with local processing power to run local applications. This concept can be carried over to other software packages, with specialized programs—such as a CAD package being stored on specific server machines and being made accessible to multiple clients. Some machines would be client sites only.

The concept of client/server architecture assumes an underlying framework that consists of many PCs and workstations as well as a smaller number of mainframe machines, connected via LANs and other types of computer networks. A client in this framework is typically a user machine that provides user interface capabilities and local processing. When a client requires access to additional functionality—such as database access—that does not exist at that machine, it connects to a server that provides the needed functionality.

A server is a system containing both hardware and software that can provide services to the client machines, such as file access, printing, archiving, or database access.



15.Explain any four types of attributes with example.

Attributes are the data elements of a table which are used to describe the characteristics of a table and provide information about the real-world entities that the table represents. Attributes are also referred to as fields or columns. Attributes are the basic components of relational databases and are necessary for a database to be valid.

1. **Simple Attribute:** A simple attribute is one that contains only a single value. For example, an employee table might have an attribute called 'name' which contains the name of the employee.

2. **Composite Attribute:** A composite attribute is made up of two or more simple attributes. For example, an address attribute might be composed of two simple attributes: city and state.

3. **Single-valued Attribute:** A single-valued attribute is one that contains only one value at a time. For example, an employee table might have an attribute called 'salary' which contains the salary of the employee.

4. **Multivalued Attribute:** A multivalued attribute is one that contains more than one value at a time. For example, an employee table might have an attribute called 'skills' which contains the list of skills that the employee has.

In summary, attributes are the data elements of a table which are used to describe the characteristics of a table and provide information about the real-world entities that the table represents. They can be simple, composite, single-valued, or multivalued, depending on the specific use case.

16. Write a short note on Entity types and Entity sets.

Entity Type:

The entity type is a collection of the entity having similar attributes. We have each row as an entity and they are having common attributes i.e each row has its own value for attributes. An entity type in an ER diagram is defined by a name and a set of attributes. The table below shows how the data of different entities(different students) are stored.

Student				Entity Type
Roll_no	Student_name	Age	Mobile_no	
1	Andrew	18	7089117222	
2	Angel	19	8709054568	Entity
3	Priya	20	9864257315	
4	Analisa	21	9847852156	

So, we can define the above STUDENT table as an entity type because it is a collection of entities having the same attributes.

We use a rectangle to represent an entity type in the E-R diagram, not entity.



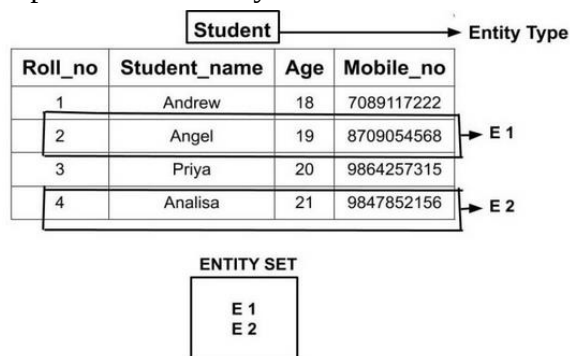
Types of Entity type:

(i) **Strong Entity Type:** Strong entity are those entity types which has a key attribute.

(ii) **Weak Entity Type :** Weak entity type doesn't have a key attribute.

Entity Set:

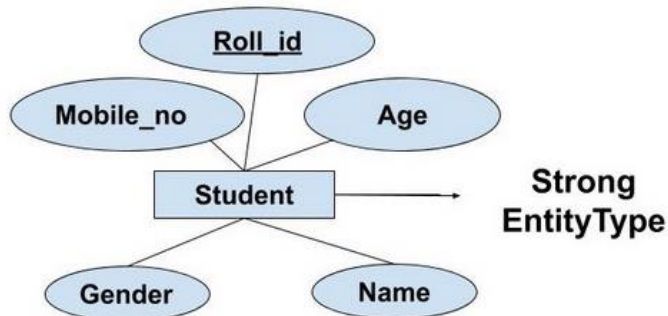
Entity Set is a collection of entities of the same entity type. We can say that entity type is a superset of the entity set as all the entities are included in the entity type. Let's try to understand this with the help of an example.



In the above example of STUDENT entity type, a collection of entities from the Student entity type would form an entity set.

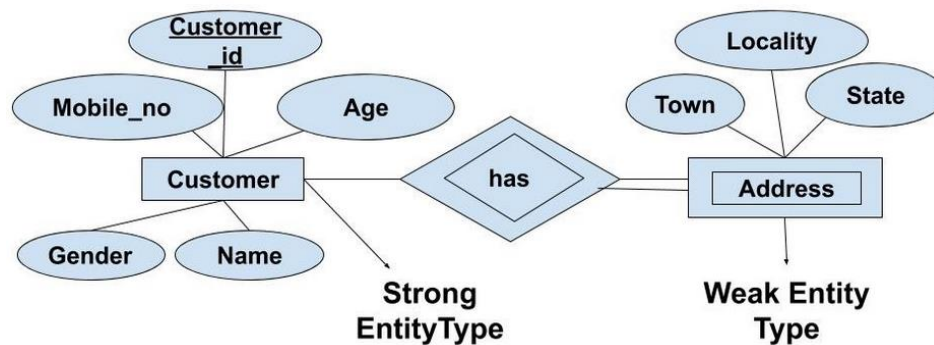
17. Explain Strong and weak entity types in DBMS with example.

Strong Entity Type: Strong entity are those entity types which has a key attribute. The primary key helps in identifying each entity uniquely. It is represented by a rectangle. In the example, Roll_no identifies each element of the table uniquely and hence, we can say that STUDENT is a strong entity type.



Weak Entity Type : Weak entity type doesn't have a key attribute. Weak entity type can't be identified on its own. It depends upon some other strong entity for its distinct identity. This can be understood with a real-life example. There can be children only if the parent exists. There can be no independent existence of children.

A weak entity is represented by a double outlined rectangle. The relationship between a weak entity type and strong entity type is called an identifying relationship and shown with a double outlined diamond instead of a single outlined diamond. This representation can be seen in the diagram below.



18. Why constraints are used in Relationship types

- Constraints are used in relationship types in DBMS to ensure that data is stored in a consistent and logical way. They help to ensure the integrity of the data by setting rules for the data and how it can be stored and manipulated.
- Constraints can be used to define the types of relationships between different tables in a database. For example, a foreign key constraint can be used to define the relationship between a table and its related table, such as a one-to-many or many-to-many relationship. A foreign key constraint ensures that any data inserted into the table is related to the data in the related table. This type of constraint helps to ensure data consistency and accuracy across the database.
- In addition, constraints can be used to enforce referential integrity. This means that any data inserted into a table must be related to data in other related tables. This helps to ensure that data is not inserted into the wrong table, as well as preventing data from being deleted from a table that is referenced by other tables.
- Constraints can also be used to enforce business rules. For example, a constraint may be used to ensure that data is entered in a certain format or is within a certain range. This ensures that data is accurate and consistent across the database.
- Constraints are also used to enforce data security. For example, a constraint may be used to ensure that only certain types of users can access certain types of data. This ensures that sensitive data is not accessed by unauthorized users.
- Finally, constraints can be used to limit the amount of data that can be stored in a table. This helps to ensure that the database does not become too large and slow down performance.

19. Explain Relationship Type and cardinality ratio with an example

A relationship type is the relationship between two tables. It can be of three types: one-to-one, one-to-many and many-to-many.

A one-to-one relationship is when an entry in one table has only one corresponding entry in another table. An example of this relationship is the relationship between a student and their teacher, where each student has only one teacher and vice versa.

A one-to-many relationship is when an entry in one table can have multiple corresponding entries in another table. An example of this relationship is the relationship between a student and their courses, where a student can take multiple courses.

A many-to-many relationship is when an entry in one table can have multiple corresponding

entries in another table and vice versa. An example of this relationship is the relationship between students and classes, where a student can take multiple classes and a class can have multiple students.

The cardinality ratio is the number of occurrences of one table to the other. It can be one-to-one, one-to-many or many-to-many and is determined by the type of relationship.

For example, in a one-to-one relationship, the cardinality ratio is one-to-one, meaning there is only one occurrence of each table.

In a one-to-many relationship, the cardinality ratio is one-to-many, meaning there can be multiple occurrences of one table and one occurrence of the other.

In a many-to-many relationship, the cardinality ratio is many-to-many, meaning there can be multiple occurrences of both tables.

In conclusion, understanding the relationship type and cardinality ratio is essential for creating and managing a database. It helps in determining the way data is organized in a database and ensures that the data is logically organized.

20. Illustrating with an example explain the term entity, attributes and relationships.

Entity:

Entity in DBMS can be a real-world object with an existence, For example, in a College database, the entities can be Professor, Students, Courses, etc. Entities has attributes, which can be considered as properties describing it, for example, for Professor entity, the attributes are Professor_Name, Professor_Address, Professor_Salary, etc. The attribute value gets stored in the database.

Example of Entity in DBMS

<Professor>

Professor_ID	Professor_Name	Professor_City	Professor_Salary
P01	Tom	Sydney	\$7000
P02	David	Brisbane	\$4500
P03	Mark	Perth	\$5000

Here, Professor_Name, Professor_Address and Professor_Salary are attributes. Professor_ID is the primary key

Types of DBMS Entities

Strong Entity: The strong entity has a primary key. Weak entities are dependent on strong entity. Its existence is not dependent on any other entity.

Weak Entity: The weak entity in DBMS do not have a primary key and are dependent on the parent entity. It mainly depends on other entities.

Attributes:

Attributes define the properties of a data object. The attribute is a quality or characteristic that defines a person, group, or data objects. It is actually the properties that define the type of entity. An attribute can have a single or multiple or range of values as per our needs.

There are three types of attributes:

1. Naming attributes –

To name an instance of a data object, naming attributes are used. User naming attributes identify user objects such as Login_names and User_Id for some security purpose.

2. Descriptive attributes –

These attributes are used to describe the characteristics or features or the relationship of the data object. Sometimes also referred to as relationship attributes.

3. Referential attribute –

These are the attributes that are used to formalize binary and associative relationships and in referring to another instance in another table.

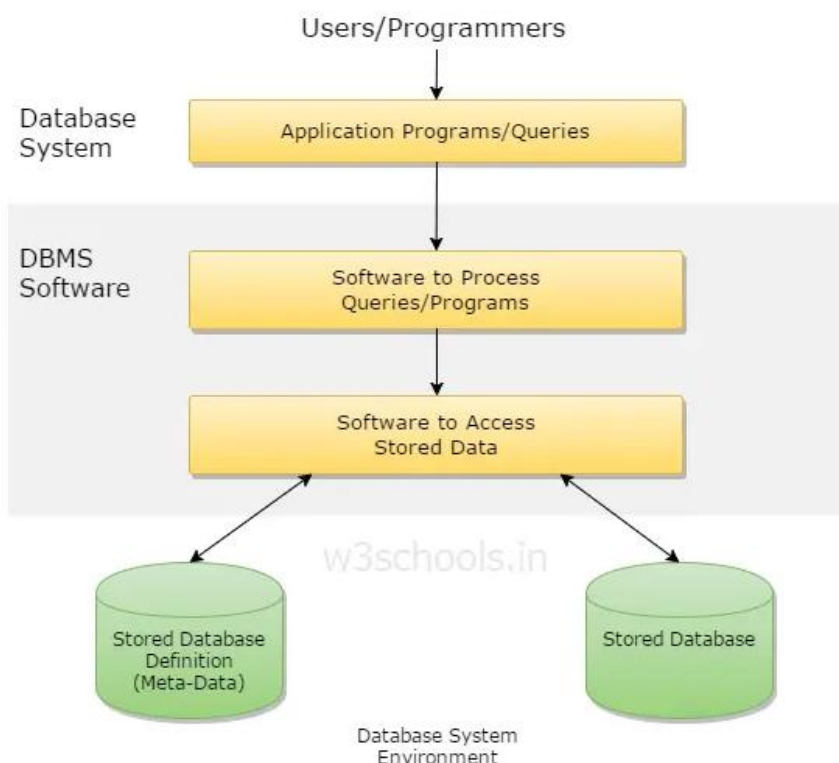
Relationship:

Relationship contains entities and relationships. ER model enables us to know how these entities are associated with each other. Entities interact with other entities through associations or relationships.

21. Explain the Database System Environment.

A database environment is a collective system of components that comprise and regulates the group of data, management, and use of data, which consist of software, hardware, people, techniques of handling database, and the data also.

Here, the hardware in a database environment means the computers and computer peripherals that are being used to manage a database, and the software means the whole thing right from the operating system (OS) to the application programs that include database management software like M.S. Access or SQL Server. Again the people in a database environment include those people who administrate and use the system. The techniques are the rules, concepts, and instructions given to both the people and the software along with the data with the group of facts and information positioned within the database environment.



22.Explain the different types of Database Management System.

(i) Relational Database

- A relational database management system (RDBMS) is a system where data is organized in two-dimensional tables using rows and columns.
- This is one of the most popular data models which is used in industries. It is based on SQL.
- Every table in a database has a key field which uniquely identifies each record.
- This type of system is the most widely used DBMS.
- Relational database management system software is available for personal computers, workstation and large mainframe systems.
- For example – Oracle Database, MySQL, Microsoft SQL Server etc.

Std ID	Name	City
201	Bob	Hyderabad
204	Lucky	Chennai
205	Pinky	Bangalore

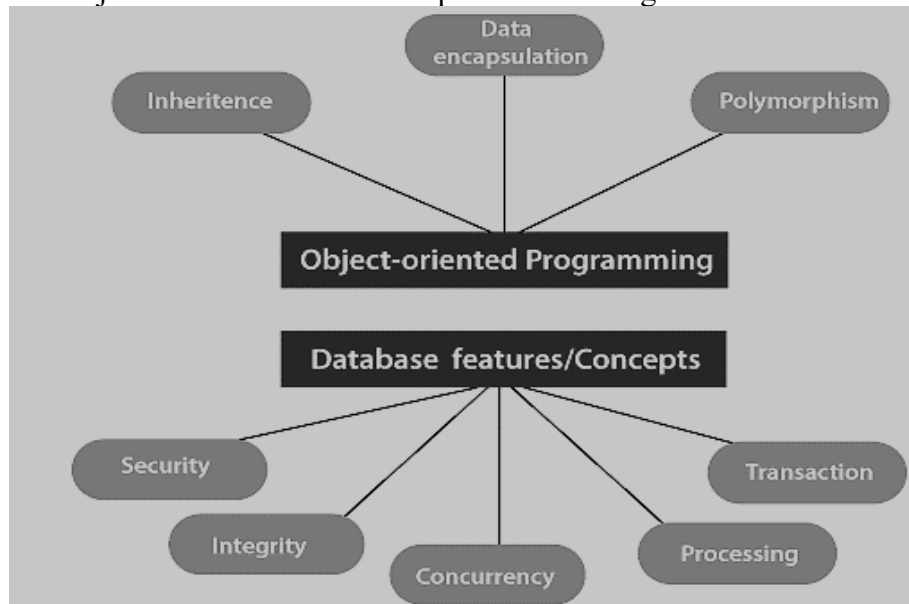
(ii) Object Oriented Database

It is a system where information or data is represented in the form of objects which is used in object-oriented programming.

- It is a combination of relational database concepts and object-oriented principles.
- Relational database concepts are concurrency control, transactions, etc.
- OOPs principles are data encapsulation, inheritance, and polymorphism.
- It requires less code and is easy to maintain.

For example – Object DB software.

The object oriented database is represented in diagram format below –



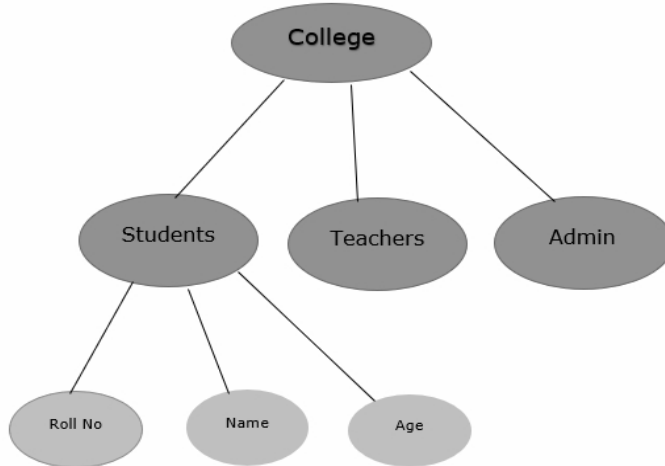
(iii) Hierarchical Database

It is a system where the data elements have a one to many relationship (1: N). Here data is organized like a tree which is similar to a folder structure in your computer system.

- The hierarchy starts from the root node, connecting all the child nodes to the parent node.
- It is used in industry on mainframe platforms.

For example– IMS(IBM), Windows registry (Microsoft).

An example of a hierarchical database is given below –



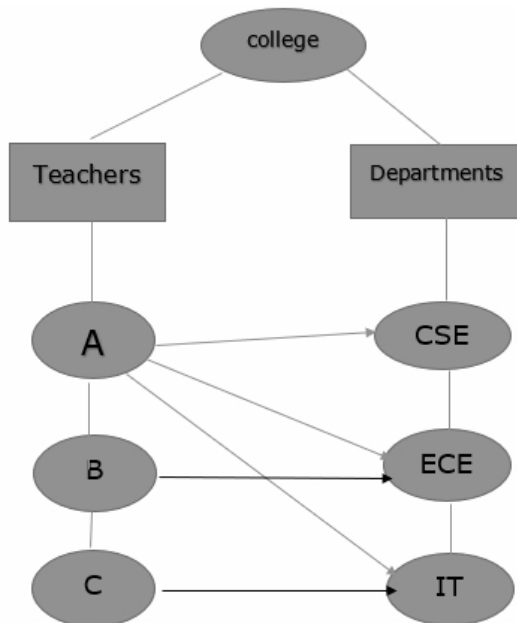
(iv) Network database

A Network database management system is a system where the data elements maintain one to one relationship (1: 1) or many to many relationship (N: N).

It also has a hierarchical structure, but the data is organized like a graph and it is allowed to have more than one parent for one child record.

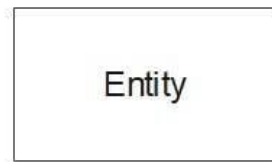
Example

Teachers can teach in multiple departments. This is shown below –



23. Describe the various symbols used in an ER diagram using example.

Symbol	Name	Description
--------	------	-------------



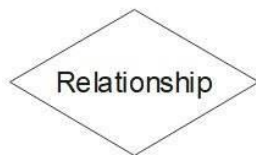
Entity

This is a basic entity that is represented by a rectangle with its name inside.



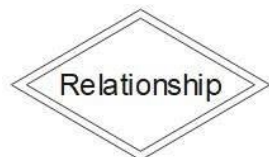
Weak Entity

This is an entity that can't solely be identified with its attributes (due to the absence of a primary key). It inherits the identifier of its parent entity and often integrated it with a partial key.



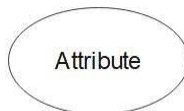
Strong Relationship

A strong relationship is depicted by a single rhombus with its name inside. In this, an entity is independent – that is, its primary key for any child doesn't contain the primary key of the linked entity.



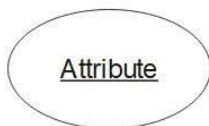
Weak or Identifying Relationship

A weak relationship is depicted by a double rhombus with the name inside. In this, the child is dependent on the parent entity as its primary key would contain a component of the parent's primary key.



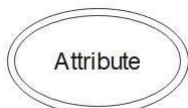
Attribute

A basic attribute is represented by a single oval with its name written inside.



Key Attribute

This is a special attribute that is used to uniquely identify an entity. It is represented by an oval with its name underlined.



Multi-valued Attribute

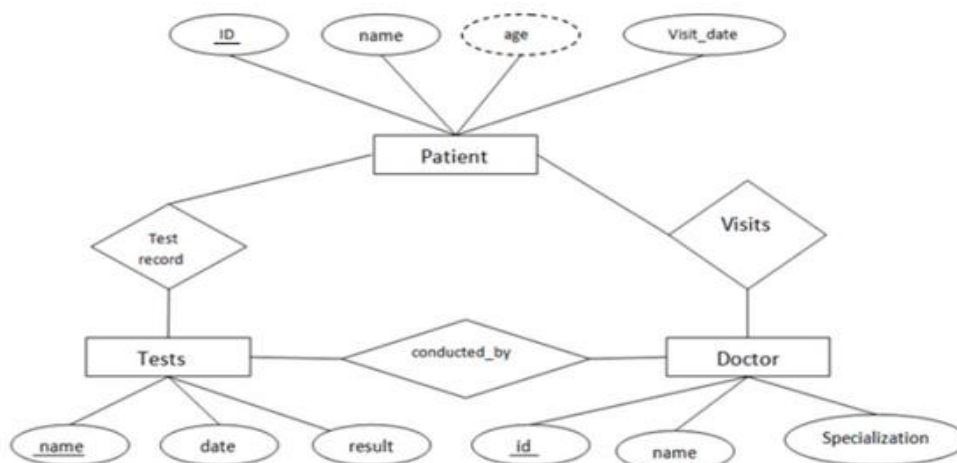
These are the attributes that can have multiple values (like the Name attribute can have First and Last name) and are represented by a double oval.



Derived Attribute

A derived attribute might not be physically present in the database and could be logically derived from any other attribute (represented by a dotted oval).

Example: Hospital ER Model



This is an ER model of a Hospital. The entities are represented in rectangular boxes and are Patient, Tests and Doctor.

Each of these entities have their respective attributes which are –

- **Patients** - ID(primary key), name, age, visit_date
- **Tests**- Name(primary key), date, result
- **Doctor**- ID(primary key), name, specialization

The relationships between different entities are represented by a diamond shaped box.

24. Explain the various types of attributes used in ER modelling.

Attribute is a particular property that describes the entity. For example, employee entity may be described by employee name, age, address, salary and job etc. A particular entity will have a value for each attribute.

Example:

The attributes of student entity are as follows –

- Roll number
- Name
- Branch
- Age

Types of Attributes:

(i) Composite attribute

It can be divided into smaller sub parts, each sub part can form an independent attribute.

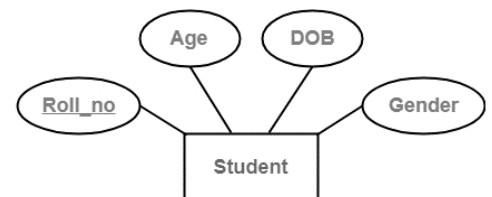
For example –

Name
FirstName MiddleName LastName

(iii) Single valued Attribute

Attributes having a single value for a particular item is called a single valued attribute.

For example: Room Number

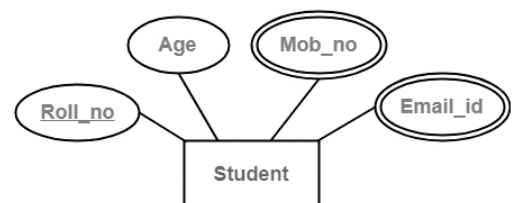


(iv) Multi-valued Attribute

Attribute having a set of values for a single entity is called a multi-valued attribute.

For example –

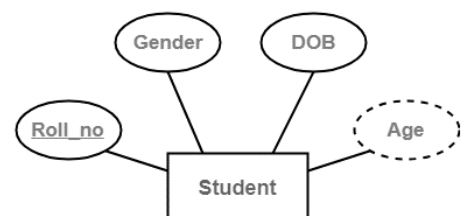
e-mail
Tel.No
Hobbies



(iv) Derived Attributes or stored Attributes

When one attribute value is derived from the other is called a derived attribute.

For example: Age can be derived from date of birth, where,
Age is the derived attribute.
DOB is the stored attribute.



(v) Complex Attribute

Nesting of composite and multi-valued attributes forms a complex attribute.

For example-

If a person has more than one house and each house has more than one phone. Then, that attribute phone is represented as a complex attribute.

(vi) Null Values: In some cases, a particular entity may not have an applicable value for an attribute. For example, degrees of a person. If a person is not having any degree, this attribute is not applicable to him. For such situations, a special value called Null value can be created. Null value can also be used if we do not know the value of an attribute for a particular entity.

25.What is participation role? When is it necessary to use role names in the description of relationship types?

Participation role is a term used in database design to describe the role that an entity plays in a relationship. For example, an employee entity could have a participation role of “employer” in a relationship where the employee is employed by a company.

It is necessary to use role names in the description of relationship types when there is a need to distinguish between multiple entities in the same relationship. This is especially important when the same type of entity is involved in multiple relationships. Using role names can help to clarify the meaning of the relationship and make it easier to understand the underlying data model.

26.Drawing of ER model of university database application considering the constraints –

- ☐ **A university has many departments.**
- ☐ **Each department has multiple instructors (one person is HOD). Here the HOD refers to the head of department.**
- ☐ **An instructor belongs to only one department.**
- ☐ **Each department offers multiple courses, each subject is taught by a single instructor.**
- ☐ **A student may enroll for many courses offered by different departments.**

Step 1 – Identifying the entity sets.

The entity set has multiple instances in a given business scenario.

As per the given constraints the entity sets are as follows –

Department
Course
Student
Instructor

Head of the Department (HOD) is not an entity set. It is a relationship between the instructor and department entities.

Step 2 – Identifying the attributes for the given entities

Department – the relevant attributes are department Name and location.

Course – The relevant attributes are courseNo, course Name, Duration, and prerequisite.

Instructor – The relevant attributes are Instructor Name, Room No, and telephone number.

Student – The relevant attributes are Student No, Student Name, and date of birth.

Step 3 – Identifying the Key attributes

Department Name is the key attribute for Department.

CourseNo is the key attribute for Course entity.

Instructor Name is the key attribute for the Instructor entity.

StudentNo is the key attribute for Student entities.

Step 4 – Identifying the relationship between entity sets

The department offers multiple courses and each course belongs to only one department, hence cardinality between department and course is one to many.



One course is enrolled by multiple students and one student for multiple courses. Hence, relationships are many to many.



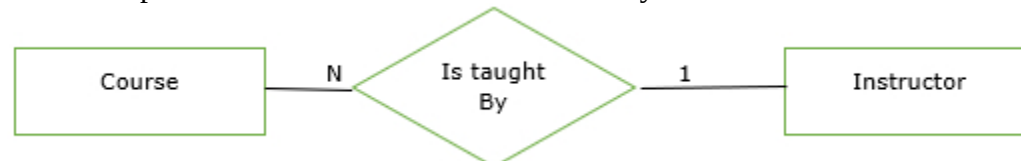
One department has multiple instructors and one instructor belongs to one and only one department, hence the relationship is one to many.



Each department has one “HOD” and one instructor is “HOD” for only one department, hence the relationship is one to one. Here, HOD refers to the head of the department.



One course is taught by only one instructor but one instructor teaches many courses hence the relationship between course and instructor is many to one.

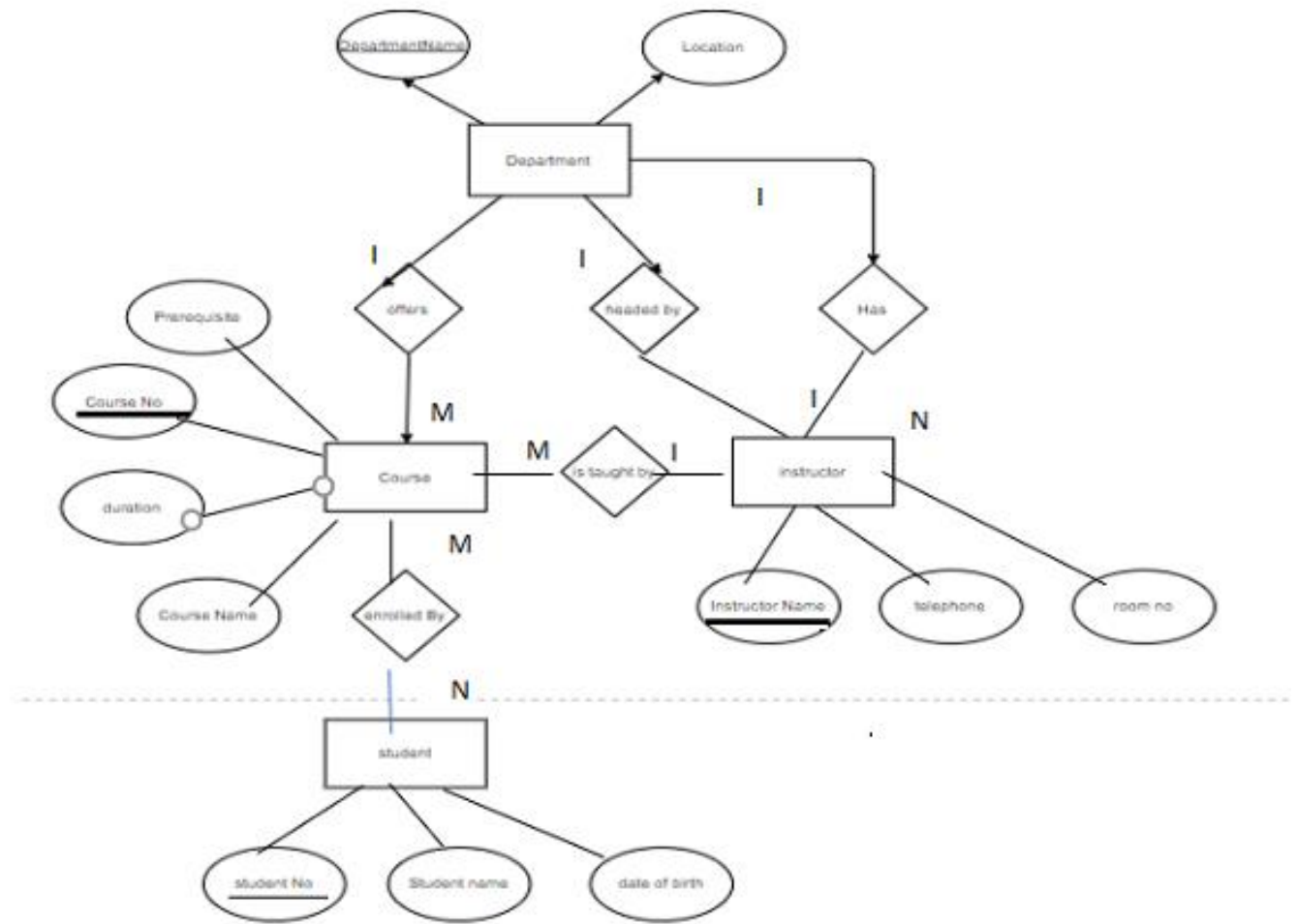


The relationship between instructor and student is not defined because of the following reasons –

- There is no significance in the relationship.
- We can always derive this relationship indirectly through course and instructors, and course and student.

Step 5 – Complete ER model

The complete ER Model is as follows –



27. Draw an ER model of the Banking database application considering the following constraints –

- ☐ A bank has many entities.
- ☐ Each customer has multiple accounts.
- ☐ Multiple customers belong to a single branch.
- ☐ Single customer can borrow multiple loans.
- ☐ A branch has multiple employees.

Step 1 – Identify the entity sets

The entity set has multiple instances in a given business scenario.

As per the given constraints the entity sets are as follows –

Customer
Account
Account
Loan
Loan

Step 2 – Identify the attributes for the given entities

Customer – the relevant attributes are customerName, CustomerID, address.

Account – The relevant attributes are AccountNo, balance.

Branch – The relevant attributes are branchID, branchName, address.

Loan – The relevant attributes are loanNo, paymentMode, dateOfLoan, and amount.

Employee – The relevant attributes are empID, empName, dateOfJoin, experience, qualification.

Step 3 – Identify the Key attributes

CustomerID is the key attribute for a customer.

AccountNo is the key attribute for Account entities.

BranchID is the key attribute for branch entities.

LoanNo is the key attribute for a loan entity.

EmpID is the key attribute for an Employee entity.

Step 4 – Identify the relationship between entity sets

One customer is enrolled by multiple accounts and one account for multiple customers. Hence, the relationship is many to many.



Many customers belong to one branch but one branch belongs to many customers. Hence, the relationship between customer and branch is many to one.



One customer can borrow multiple loans in the same way multiple loans can borrow a single customer, hence the relationship between customer and loan is one to many.

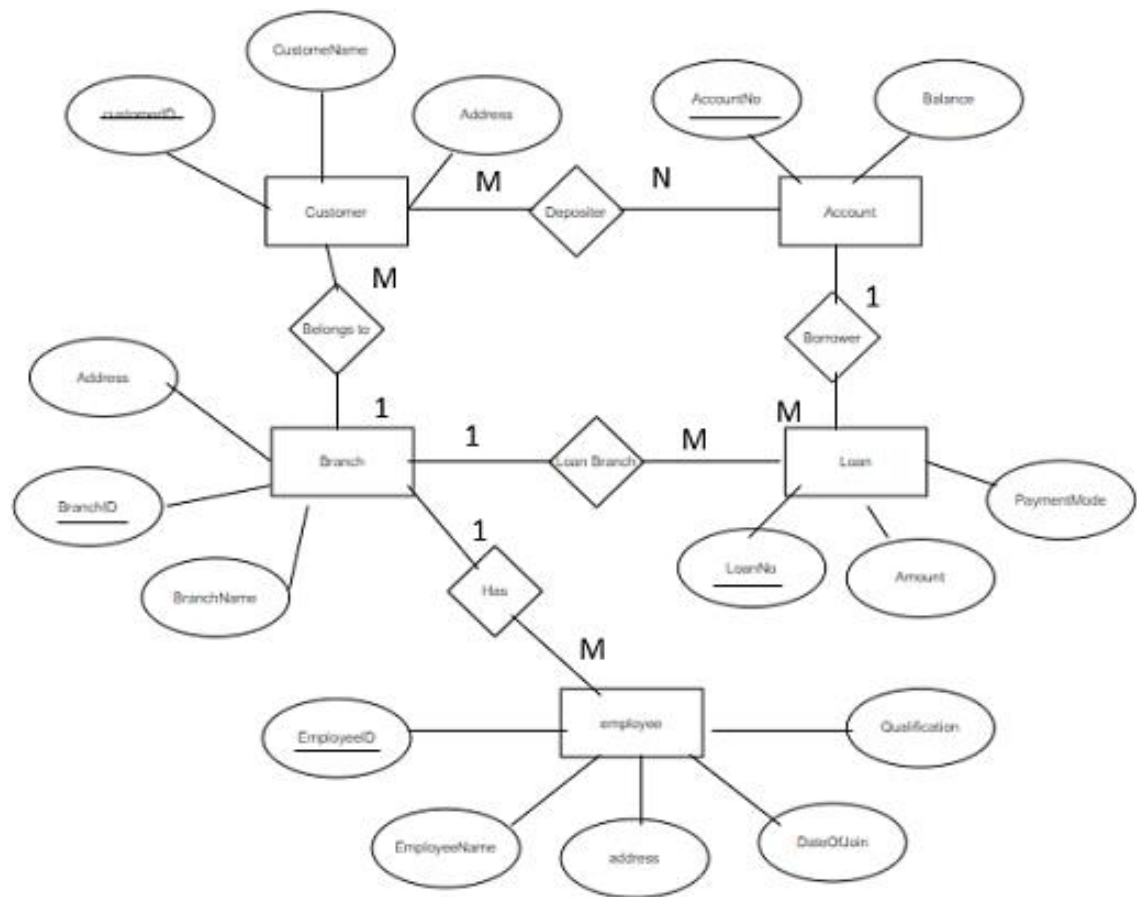


One branch has many employees and in the same way the number of employees works in a single branch.



Step 5 – Complete ER diagram

The complete ER diagram is as follows –



28. Draw an ER model for library management application considering the following constraints –

- ☐ In a library multiple students can enroll.
- ☐ Students can become a member by paying an appropriate fee.
- ☐ The books in the library are identified by a unique ID.
- ☐ Students can borrow multiple books from subscribed libraries.

Step 1 – Identify the entity sets

The entity set has multiple instances in a given business scenario.

As per the given constraints the entity sets are as follows –

Book Publisher Member Section Granter

Step 2 – Identify the attributes for the given entities

Book – The relevant attributes are title, author, price, Isbn.

Member – The relevant attributes are Name, Bday, MID, address, phone, age.

Section – The relevant attributes are Sid, name, phone.

Publisher – The relevant attributes are name, phone, Pid, address.

Granter – The relevant attributes are phone, name, Nic, post, address.

Step 3 – Identify the Key attributes

Sid is the key attribute for the section.

Mid is the key attribute for member entities.

Isbn is the key attribute for a book entity.

Pid is the key attribute for a publisher entity.

Nic is the key attribute for a granter entity.

Step 4 – Identify the relationship between entity sets

Multiple books are arranged in a single section and one section has multiple books. Hence, the relationship between book and section is many to one.



One member borrows multiple books and multiple books can borrow a single person. Hence, the relationship between member and book is one-to-many.



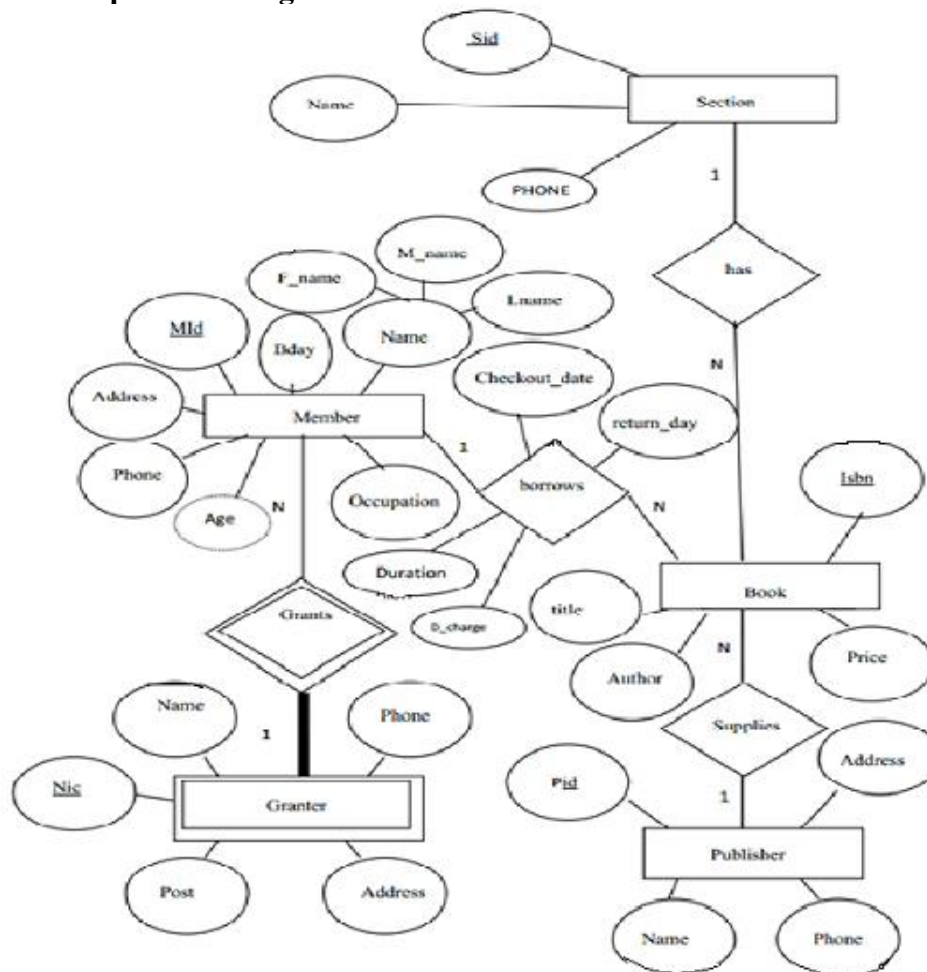
One publisher can supply multiple books and multiple books can be supplied by a single publisher. Hence, the relationship between publisher and book is one-to-many.



One granter can grant multiple members and multiple members can grant a single granter. Hence, the relationship between granter and member is one-to-many.



Step 5 – Complete ER diagram



29. Draw of ER model for company considering the following constraints –

- ☐ **In a company, an employee works on many projects which are controlled by one department.**
- ☐ **One employee supervises many employees.**
- ☐ **An employee has one or more dependents.**
- ☐ **One employee manages one department.**

Step 1 – Identify the entity sets

The entity set has multiple instances in a given business scenario.

As per the given constraints the entity sets are as follows:

Employee
Department
Project
Dependent

Step 2 – Identify the attributes for the given entities

Employee – The relevant attributes are name, ssn, sex, address, salary.

Department – The relevant attributes are Name, number of employees, location.

Project – The relevant attributes are number, name, location.

Dependent – The relevant attributes are name, sex, birth date, relationship.

Step 3 – Identify the Key attributes

SSN is the key attribute for Employee.

Number is the key attribute for the Department.

Number is the key attribute for Project.

Name is the key attribute for a dependent entity.

Step 4 – Identify the relationship between entity sets

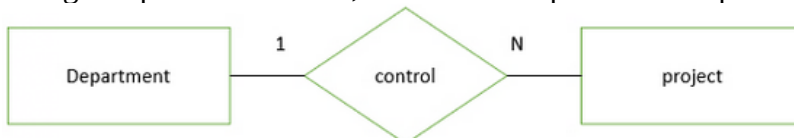
Multiple employees work for a single department and one department has multiple employees. Hence, the relationship between employee and department is many to one.



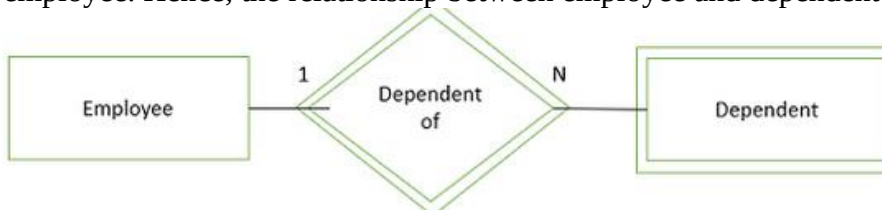
Single employee manages the entire department and one department is handled by one manager.



Each department is controlled by the number of projects and the number of projects handled by a single department. Hence, the relationship between department and project is one-to-many.



One employee dependent is of multiple dependent and multiple dependents are of single employee. Hence, the relationship between employee and dependent is one-to-many.



Dependent is a weak entity which is denoted by a double rectangle, the entity which does not contain a primary key is known as weak entity.

Step 5 – Complete ER diagram

